

Resolución Práctica 5: Clase 03

Grupo 4

Integrantes:

- Avila, Aldana, legajo 1193721, alavila@uade.edu.ar
- Caivano, Alexander Francisco, legajo 1209389, acaivano@uade.edu.ar
- Casareski, Juan Ignacio, legajo 1176109, jcasareski@uade.edu.ar
- Segura, Juan Ignacio, legajo 1242159, jsegura@uade.edu.ar

Parte A: Identificación de documentos principales

Documento / colección	Qué representa	Ejemplo	¿Por qué MongoDB?
tenants	Configuración básica de cada empresa cliente (multi-tenant)	<pre>{ "tenant_id": "empresa_acme", "name": "ACME S.A.", "status": "active", "theme": { "primary_color": "#003366" } }</pre>	Flexibilidad de esquema para configuraciones por tenant (colores, logo, etc.). Consultas simples por tenant_id.
process_definition	Definición versionada y flexible de un proceso BPM (nodos, edges, formularios dinámicos, condiciones, roles)	Ver Parte B	Estructura semi-estructurada compleja con arrays anidados (nodes/edges/fields). Cambia por tenant y versión. Ideal para embedding.
process_instances	Estado actual de una ejecución concreta de un proceso (datos del formulario, nodo actual, tarea pendiente)	Ver Parte E	Estado mutable con datos embebidos del formulario. Consultas frecuentes por tenant_id + instance_id o estado.
forms	Definiciones de formularios dinámicos reutilizables (campos, tipos, validaciones): embebidos en el nodo o referenciados	Campos dentro de nodo "form" en process_definition	Flexibilidad total de esquema (distintos campos por proceso/tenant).
tasks	Tareas humanas pendientes o en curso: embebidas como current_task en la instancia	<pre>"current_task": { "task_id": "...", "assigned_role": "compras", "status": "pending" }</pre>	Consultas por rol para listas de trabajo pendientes (con índice sobre campo anidado). Estado actual vive junto a la instancia.
Otro	No se propone colección adicional en MongoDB para este alcance	n/a	Event log masivo se deja para Cassandra (escritura append-only intensiva). Caminos complejos para Neo4j.

Respuestas a preguntas guía:

1. ¿Qué documentos tienen estructura flexible?
process_definitions (nodos/edges/forms/condiciones varían enormemente por proceso y tenant) y forms (campos dinámicos por proceso).
2. ¿Qué documentos cambian según el tenant?
Todos los principales (tenants, process_definitions, process_instances). El tenant_id es filtro principal en todas las consultas.
3. ¿Qué documentos necesitan versionado?
process_definitions (un proceso evoluciona sin romper instancias existentes que apuntan a una versión anterior).
4. ¿Qué documentos podrían crecer demasiado?
process_instances si se guarda historial completo de transiciones; el event log en particular (se deriva a Cassandra).
5. ¿Qué documentos no deberían estar en MongoDB como fuente principal?
Event log masivo (Cassandra), caminos/ciclos complejos (Neo4j), estado de baja latencia y sesiones (Redis), objetos con comportamiento encapsulado (IRIS).

Parte B: Documento process_definition

Documento JSON para el proceso “Alta de proveedor” del TPO.

```
{
  "tenant_id": "empresa_acme",
  "process_id": "alta_proveedor",
  "version": 1,
  "status": "published",
  "name": "Alta de proveedor",
  "description": "Proceso para registrar y aprobar nuevos proveedores",
  "roles": ["compras", "legal", "admin"],
  "nodes": [
    {
      "id": "start",
      "type": "start",
      "name": "Inicio"
    },
    {
      "id": "formulario_inicial",
      "type": "form",
      "name": "Formulario inicial",
      "fields": [
        { "name": "razon_social", "type": "string", "required": true },
        { "name": "cuit", "type": "string", "required": true },
        { "name": "monto_estimado", "type": "number", "required": false }
      ]
    },
    {
      "id": "revision_compras",
      "type": "task",
      "name": "Revisión de Compras",
      "assigned_role": "compras"
    },
    {
      "id": "decision_legal",
      "type": "decision",
      "name": "¿Requiere revisión legal?",
      "condition": "monto_estimado > 1000000"
    },
    {
      "id": "revision_legal",
      "type": "task",
      "name": "Revisión Legal",
      "assigned_role": "legal"
    },
    {
      "id": "end",
      "type": "end",
      "name": "Fin"
    }
  ]
}
```

```

],
"edges": [
  { "from": "start", "to": "formulario_inicial" },
  { "from": "formulario_inicial", "to": "revision_compras" },
  { "from": "revision_compras", "to": "decision_legal" },
  { "from": "decision_legal", "to": "revision_legal", "when": "true" },
  { "from": "decision_legal", "to": "end", "when": "false" },
  { "from": "revision_legal", "to": "end" }
],
"created_by": "admin_acme",
"created_at": "2026-06-01T10:00:00Z",
"published_at": "2026-06-01T12:00:00Z"
}

```

Parte C: Embedding vs referencing

Dato	¿Embedding o referencing?	Justificación	Riesgo
Nodos del proceso	Embedding	Se consultan siempre junto con la definición. Son parte integral e inseparable del proceso.	Bajo (tamaño acotado, no supera el límite de 16 MB de BSON en escenarios normales).
Conexiones entre nodos	Embedding	Íntimamente ligadas a los nodos; se consultan en conjunto para ejecutar o visualizar el flujo.	Bajo.
Campos del formulario	Embedding	Estructura dinámica y específica de cada proceso/versión. Cambia frecuentemente por tenant.	Bajo (la flexibilidad de MongoDB absorbe esta variabilidad).
Roles	Referencing	Los roles son entidades propias de cada tenant: pueden crearse, desactivarse o modificarse sin necesidad de actualizar cada proceso. Tienen ciclo de vida independiente.	Inconsistencia si se elimina un rol referenciado (se gestiona en capa de aplicación con validación previa).
Usuarios	Referencing	Tienen ciclo de vida propio (alta, baja, cambio de rol). No se consultan siempre junto con el proceso.	Inconsistencia si se elimina un usuario sin limpiar referencias (se maneja con lógica de aplicación).
Integraciones externas	Embedding (config simple) o Referencing	Si son pocas y estables, embedding. Si son muchas o cambian seguido, referencing.	Configuración desactualizada si se embebe y luego cambia en el sistema externo.
Plantillas de notificación	Referencing	Reutilizables en muchos procesos. Tienen ciclo de vida propio (se actualizan sin tocar las definiciones).	Duplicación si se embebieran en cada definición de proceso.
Reglas reutilizables	Referencing	Compartidas entre varios procesos (ej. reglas de validación de CUIT, umbrales de aprobación).	Requiere mantener referencias en la aplicación; mayor complejidad de lectura.

Respuestas a preguntas guía:

1. ¿Qué datos se consultan siempre junto con el proceso?
Nodos, edges, campos de formulario y condiciones, todos por embedding.
2. ¿Qué datos se reutilizan en muchos procesos?
Roles, plantillas de notificación y reglas reutilizables, por referencing.
3. ¿Qué datos pueden crecer demasiado?
El historial de versiones o las listas grandes de usuarios, que van por referencing o en colección separada.
4. ¿Qué datos tienen ciclo de vida propio?
Roles, usuarios, plantillas y reglas, por referencing.
5. ¿Qué datos deberían estar referenciados?
Roles, usuarios y plantillas/reglas globales.

Parte D: Versionado de procesos

Respuestas al escenario:

1. Sí, las instancias viejas deben seguir usando la versión 1 (para no interrumpir procesos en curso).
2. Sí, las nuevas instancias deben usar la versión 2 (la versión con status: "published" más reciente).
3. En process_definition: campo version (entero incremental) + campo status (draft / published / archived). Se define índice único compuesto sobre (tenant_id, process_id, version) para prevenir duplicados.
4. En process_instance: campo process_version (se graba al crear la instancia y nunca se modifica).
5. Riesgo de sobrescribir: se pierde la definición que usaban las instancias existentes, lo que impide continuar los flujos iniciados y pierde la trazabilidad.
6. Estados posibles de una definición: draft, published, archived.

Tabla de estrategias:

Estrategia de versionado	Ventaja	Riesgo
Sobrescribir documento	Simple (un solo documento por process_id)	Rompe instancias existentes. Sin rollback ni auditoría de cambios.
Crear documento por versión	Máxima claridad y aislamiento. Fácil consultar la versión exacta que usó cada instancia.	Ligera duplicación de campos estáticos (name, description), aceptable.
Mantener historial embebido	Un solo documento por process_id	El documento crece con cada versión. Difícil consultar una versión específica con eficiencia.
Usar colección separada de versiones	Separación conceptual limpia	Requiere joins lógicos en la aplicación (MongoDB no tiene joins nativos). Mayor complejidad.

Estrategia elegida: Crear documento por versión

Cada versión de un proceso vive como un documento independiente en process_definitions. El índice único (tenant_id, process_id, version) garantiza que no existan dos documentos con la misma versión para el mismo proceso y tenant.

```
db.process_definitions.createIndex(  
  { tenant_id: 1, process_id: 1, version: 1 },  
  { unique: true }  
)
```

Ventaja principal: las instancias existentes continúan con su versión original sin ningún cambio, mientras las nuevas usan la versión published actual. Permite rollback fácil publicando una versión anterior.

Parte E: Documento process_instance

```
{
  "tenant_id": "empresa_acme",
  "instance_id": "inst_1001",
  "process_id": "alta_proveedor",
  "process_version": 1,
  "status": "waiting_task",
  "current_node": "revision_compras",
  "data": {
    "razon_social": "Proveedor Demo S.A.",
    "cuit": "30-12345678-9",
    "monto_estimado": 1500000
  },
  "current_task": {
    "task_id": "task_9001",
    "assigned_role": "compras",
    "status": "pending"
  },
  "history": [
    {
      "node": "start",
      "timestamp": "2026-06-10T09:00:00Z",
      "event": "process_started"
    },
    {
      "node": "formulario_inicial",
      "timestamp": "2026-06-10T09:05:00Z",
      "event": "form_submitted"
    }
  ],
  "created_by": "usuario_123",
  "created_at": "2026-06-10T09:00:00Z",
  "updated_at": "2026-06-10T09:15:00Z"
}
```

Respuestas:

- 1. ¿Qué datos de la instancia conviene guardar en MongoDB?
Estado actual (status, current_node, current_task, data del formulario y un history mínimo de transiciones recientes). Permite consultas rápidas del estado vivo del proceso.
- 2. ¿Qué datos de la instancia podrían cachearse en Redis?
current_task (listas de trabajo por rol) y el estado resumido de instancias muy activas para baja latencia. Redis es más adecuado que MongoDB para estos accesos de alta frecuencia con TTL.
- 3. ¿Qué datos no deberían reemplazar al event log?
El historial completo de cambios, decisiones y transiciones. La instancia guarda solo el estado actual y un resumen mínimo; el log detallado (append-only, con volumen masivo) va a Cassandra.
- 4. ¿Qué información permite saber con qué versión del proceso se inició la instancia?
El campo process_version, que se graba al crear la instancia y permanece inmutable durante toda su vida útil.

Parte F: Consultas frecuentes e índices conceptuales

Consulta	Filtro principal	Colección	Índice conceptual
Obtener proceso publicado por tenant y process_id	tenant_id + process_id + status: "published"	process_definitions	{tenant_id: 1, process_id: 1, status: 1, version: -1 }
Obtener una versión específica de un proceso	tenant_id + process_id + version	process_definitions	{tenant_id: 1, process_id: 1, version: 1 } (único)

Consulta	Filtro principal	Colección	Índice conceptual
Listar procesos activos de un tenant	tenant_id + status: "published"	process_definitions	{tenant_id: 1, status: 1, published_at: -1 }
Consultar instancia por tenant e instance_id	tenant_id + instance_id	process_instances	{tenant_id: 1, instance_id: 1 } (único)
Listar instancias abiertas de un tenant	tenant_id + status (no terminal)	process_instances	{tenant_id: 1, status: 1, updated_at: -1 }
Listar tareas pendientes de un rol	current_task.assigned_role + current_task.status: "pending" + tenant_id	process_instances	{current_task.assigned_role": 1, "current_task.status": 1, tenant_id: 1 }

Se proponen 6 índices conceptuales. El índice sobre instance_id y el de (tenant_id, process_id, version) se declaran con unique: true.

Parte G: Operaciones MongoDB conceptuales

Operación	Comando conceptual	Qué demuestra
Insertar definición	db.process_definitions.insertOne({tenant_id: "empresa_acme", process_id: "alta_proveedor", version: 1, status: "draft", roles: [...], nodes: [...], edges: [...], created_by: "admin_acme", created_at: new Date() })	Creación de una nueva definición en estado draft.
Publicar versión	db.process_definitions.updateOne({tenant_id: "empresa_acme", process_id: "alta_proveedor", version: 2 }, { \$set: { status: "published", published_at: new Date() } })	Transición de estado de draft a published.
Crear instancia	db.process_instances.insertOne({tenant_id: "empresa_acme", instance_id: "inst_1002", process_id: "alta_proveedor", process_version: 2, status: "running", current_node: "start", data: {}, history: [], created_by: "usuario_456", created_at: new Date(), updated_at: new Date() })	Inicio de ejecución vinculada a una versión específica.

Operación	Comando conceptual	Qué demuestra
Actualizar estado de instancia	<code>db.process_instances.updateOne({tenant_id: "empresa_acme", instance_id: "inst_1002" }, { \$set: { status: "waiting_task", current_node: "revision_compras", current_task: { task_id: "task_9002", assigned_role: "compras", status: "pending" }, updated_at: new Date() }, \$push: { history: { node: "formulario_inicial", event: "form_submitted", timestamp: new Date() } } })</code>	Avance del flujo, actualización de tarea actual y registro mínimo en el historial.
Consultar instancias abiertas	<code>db.process_instances.find({tenant_id: "empresa_acme", status: { \$in: ["running", "waiting_task"] } }).sort({updated_at: -1 })</code>	Listado de trabajo activo por tenant.
Obtener versión publicada actual	<code>db.process_definitions.find({tenant_id: "empresa_acme", process_id: "alta_proveedor", status: "published" }).sort({version: -1 }).limit(1)</code>	Obtener la definición vigente para iniciar nuevas instancias. Se usa <code>find().sort().limit(1)</code> porque <code>findOne()</code> no admite <code>.sort()</code> .

Parte H: Límites de MongoDB dentro de FlowOps

Necesidad	¿MongoDB es suficiente?	Mejor alternativa	Justificación	Análisis CAP
Definición flexible del proceso	Sí	n/a	Esquema flexible + embedding de nodos, edges, forms y condiciones.	MongoDB CP: prioriza consistencia y tolerancia a particiones. Adecuado: las definiciones deben leerse de forma consistente.
Formularios dinámicos	Sí	n/a	Campos variables por proceso/tenant sin schema rígido.	Igual que definiciones.
Instancias actuales	Sí / Parcial	Redis (para listas de tareas de baja latencia)	MongoDB maneja el estado persistente actual. Redis para colas de trabajo de alto throughput con TTL.	MongoDB garantiza consistencia del estado; Redis sacrifica consistencia por disponibilidad y latencia.

Necesidad	¿MongoDB es suficiente?	Mejor alternativa	Justificación	Análisis CAP
Auditoría masiva de eventos	No	Cassandra / ScyllaDB	Escritura intensiva append-only y consultas por rango de tiempo/tenant. MongoDB no está optimizado para este patrón.	Cassandra AP: alta disponibilidad y tolerancia a particiones. Consistencia eventual aceptable para eventos de auditoría.
Caminos complejos y ciclos	No	Neo4j	Traversals, detección de ciclos y caminos óptimos son naturales en grafos. MongoDB solo puede emularlos con recursión manual.	Neo4j CA: prioriza consistencia y disponibilidad en clúster; no está pensado como sistema distribuido tolerante a particiones.
Estado con baja latencia	Parcial	Redis	Consultas de tareas pendientes por rol requieren latencia sub-milisegundo en escenarios de alto tráfico.	Redis AP: sacrifica consistencia fuerte por disponibilidad y baja latencia.
Sesiones de usuario	No	Redis	TTL nativo, alta rotación y baja latencia. MongoDB no tiene TTL automático por campo sin índice especial.	Redis AP: ideal para datos efímeros con expiración automática.
Objetos con métodos complejos	No	IRIS	Cuando se requiera modelar objetos de dominio con comportamiento encapsulado y reglas de negocio complejas.	IRIS provee un modelo orientado a objetos con lógica embebida, fuera del ámbito del teorema CAP tradicional.

Respuestas:

1. ¿Por qué MongoDB no debería ser la única base del TPO?
FlowOps tiene patrones de acceso muy distintos: definiciones flexibles (MongoDB), escritura masiva append-only de eventos (Cassandra), traversals de flujo (Neo4j), estado de baja latencia (Redis). Una sola base obligaría a compromisos que degradan rendimiento o mantenibilidad. Además, el TPO exige al menos dos modelos NoSQL distintos.
2. ¿Qué dato dejarían para Cassandra?
El event log / auditoría masiva de eventos (inicio, transiciones, decisiones, fin), particionado por (tenant_id, instance_id) y ordenado por created_at.
3. ¿Qué consulta dejarían para Neo4j?
Consultas de caminos alternativos entre nodos, validación de flujos con ciclos o detección de caminos inalcanzables en la definición de un proceso.
4. ¿Qué dato dejarían para Redis?

Listas de tareas pendientes por rol (current_task) y estado efímero de instancias activas (cache de alta rotación con TTL).

5. ¿Qué situación podría justificar IRIS?

Cuando se requiera modelar objetos de dominio con comportamiento encapsulado, herencia y métodos de negocio que van más allá de datos puros; por ejemplo, un objeto Proveedor con lógica de validación interna.

Parte I: Integración con la arquitectura políglota

Subsistema FlowOps	Modelo / tecnología	Rol
Definiciones de procesos	MongoDB	Almacenar y versionar definiciones flexibles (nodos, formularios dinámicos, condiciones, roles) con embedding. Consultas por tenant_id + process_id + versión.
Instancias	MongoDB (principal) + Redis (cache caliente)	Estado actual de ejecuciones, datos del formulario y tarea pendiente en MongoDB. Redis para listas de trabajo de baja latencia y cache del estado activo.
Eventos de auditoría	Cassandra / ScyllaDB	Log masivo append-only de eventos por (tenant_id, instance_id), ordenado por created_at. Alta escritura, consultas temporales. Consistencia eventual aceptable.
Flujo y caminos	Neo4j	Representación como grafo para traversals, validación de caminos y detección de ciclos en la definición del proceso.
Cache / sesiones	Redis	Estado efímero, colas de tareas por rol y TTL de sesiones/configuración caliente. Baja latencia, alta disponibilidad.
Objetos complejos	IRIS / opcional	Modelado de entidades de dominio con comportamiento cuando sea necesario (ej. objetos Proveedor con lógica embebida).
Métricas	Multidimensional / opcional	Dashboards de tiempos de ciclo, volumen de procesos y SLA por tenant.